CERTE · COEVORDEN · 19-22 OCT 2026

Building agents with orq.ai

Four days on site. Platform first, then agents, evals and your own use cases.

4

DAYS · ON SITE

Bauke Brenninkmeijer · orq.ai

Bauke Brenninkmeijer

Your trainer for the technical days, from orq.ai.



HOW WE WORK

Hands-on

Short explanations, then you do it in Studio, the CLI or code.

Questions any time, and 5 minutes after every practical.

Today and tomorrow

Day 1 is the platform. Day 2 is agents, then a hackathon on your use cases.

DAY 1 Mon 19 Oct · Platform and IT controls

MORNING · 09:00

- Platform overview and set-up
- Platform options and agent basics
- Model access: Europe and ZDR — EXERCISE

12:00 · LUNCH

AFTERNOON

- Gateway resilience and tracing
- Keys, roles and audit logs — EXERCISE
- Budgets and spend
- Guardrails and PII — EXERCISE

DAY 2 Tue 20 Oct · Agents, then build

MORNING · 09:00

- Build a generic agent
- Knowledge bases and retrieval — EXERCISE
- MCP gateway: expose and hide tools — EXERCISE
- Build from your coding assistant

12:00 · LUNCH

AFTERNOON · HACKATHON

- Kickoff: your use cases
- Build: your first agents
- Evals and LLM-as-judge
- Demos and review

> Each exercise is 10-15 minutes, followed by 5 minutes for questions.

Day 1: the platform

Only what the master deck does not already cover.

01 **The platform map**
How Studio, CLI and evaluatorq relate, and where orq.ai runs.

02 **Tools, loops and MCP in orq.ai**
Who runs a tool, who owns the loop, and why MCP goes through a gateway.

03 **Models and the gateway**
Model Garden, EU and ZDR filters, routing, fallbacks.

04 **IT controls**
Roles, SSO, keys, audit logs, budgets, usage, guardrails, traces.

05 **MCP, knowledge, agents and CLI**
The pieces Day 2 builds on.

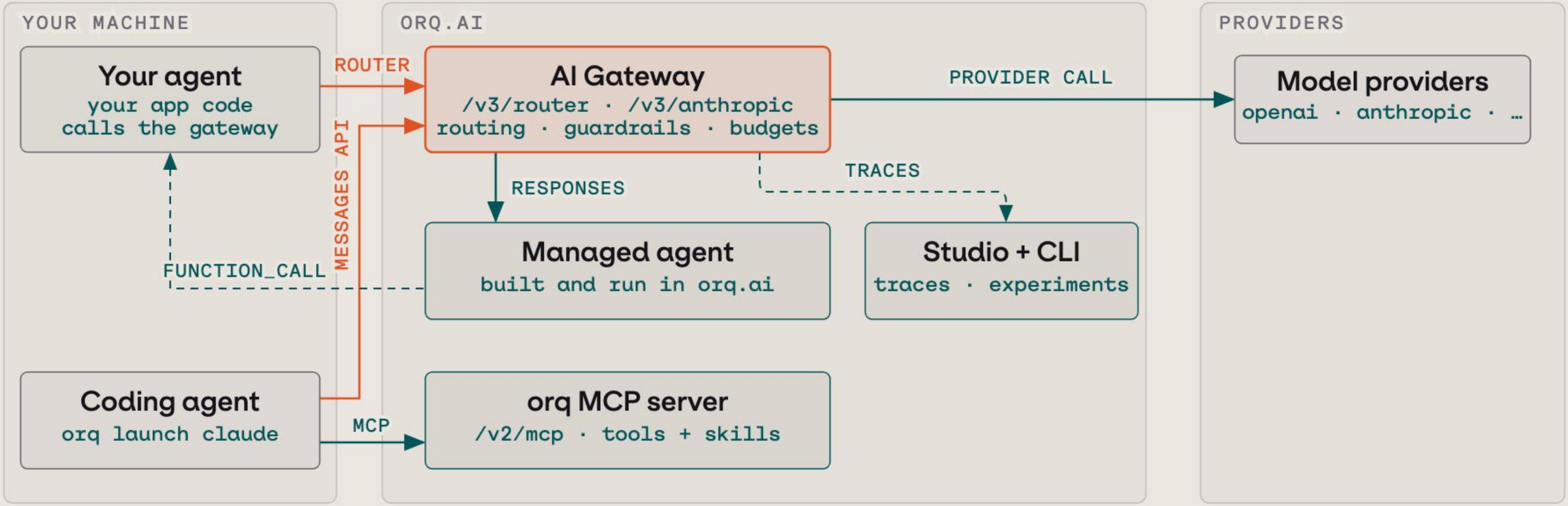
Already in the master deck

Not repeated here. Open the master deck for these.

TOPIC	MASTER DECK SLIDES
LLM basics, automation vs workflow vs agent	10-11
Tools	12-14
Model parameters	15-27
Orqflow, Smart Router, observability, governance	29-37
Guardrails and PII redaction	68-75
Evaluators, experiments, annotation	76-85, 102
Agents and sub-agents	95-101

Platform map: how the pieces connect

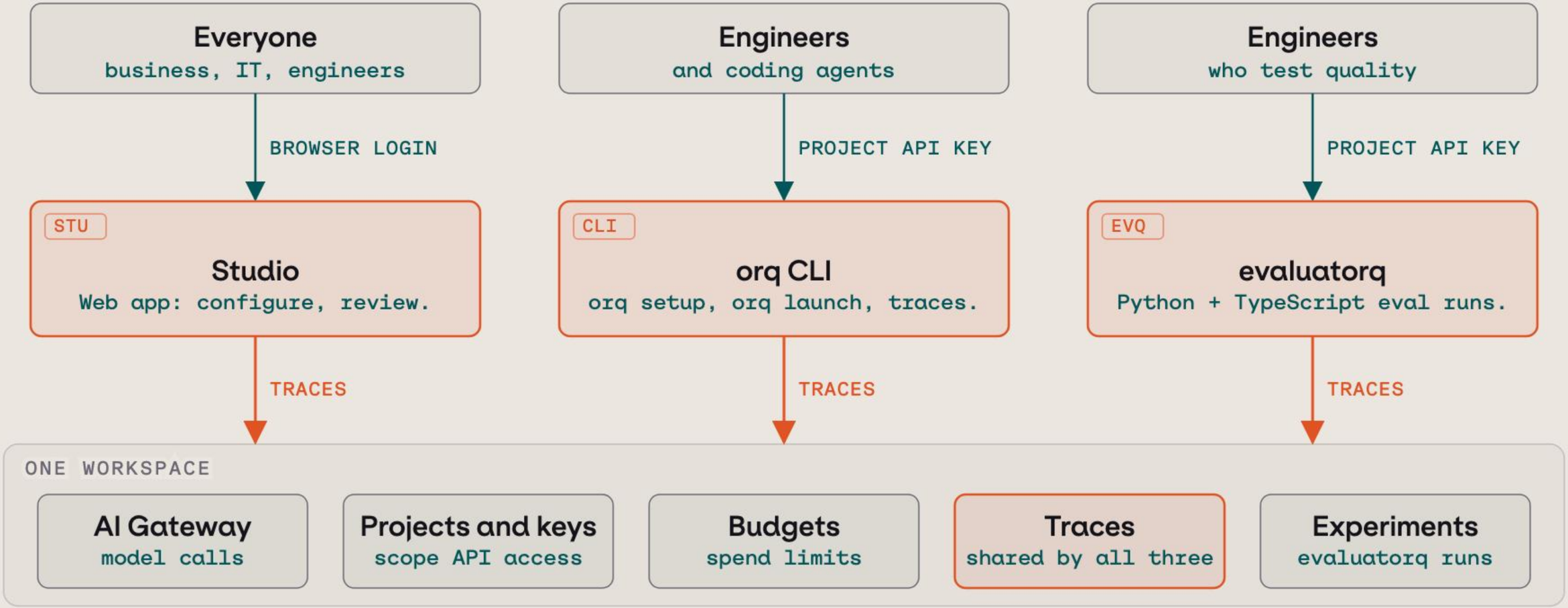
Your code and your coding agent both call the same gateway.



orange = call into the AI Gateway · solid = internal call · dashed = return or passive

Platform map: who uses which tool

One workspace and shared traces; Studio login and API keys serve different jobs.



The Studio sidebar

Five folders. Closed on the left, opened on the right.

STUDIO · FOLDERS CLOSED

- Home
- Search ⌘ K
- Marketplace
- Settings

PERSONAL

- Favorites
- Recents

PLATFORM

- AI Gateway ›
- Observability ›
- Managed Agents ›
- Optimizations ›
- Prompt Management ›

OPENED · AI GATEWAY

PLATFORM

- AI Gateway ⌄
  - Models
  - Smart Router
  - BYOK
  - Routing Rules
  - Guardrail Rules
  - MCP Portal Beta

OPENED · OBSERVABILITY

- Observability ⌄
  - Traces
  - Logs Beta
  - Threads
  - Automations
  - Identities
  - Monitors Beta
  - Alerts Beta

OPENED · MANAGED AGENTS

- Managed Agents ⌄
  - Agents
  - Deployments
  - Skills
  - Tools
  - Memories
  - Knowledge Bases

OPENED · LAST TWO

- Optimizations ⌄
  - Datasets
  - Evaluators
  - Experiments
  - Annotation Queues
  - Annotations
- Prompt Management ⌄
  - Prompts
  - Playground

Four capability areas

A mental map of what the platform does; menu labels may differ.

MANAGED AGENTS
Build
Tools · Skills · Memory · Prompts · Knowledge bases

→

AI GATEWAY
Route every call
Models · Smart Router · Routing Rules · BYOK · Budgets

→

PROVIDERS
Model call

↑ traces ↑ traces

AI OBSERVABILITY
Traces · Threads · Monitors · Alerts · Insights · Automations · Datasets · Evaluators · Experiments

GOVERNANCE
Guardrail Rules · Identities · MCP Portal · API keys + SSO

Where orq.ai runs

Choose where the platform runs, then choose approved model routes.

OPTION 1

EU SaaS

orq.ai hosts the platform in the EU.

Fastest start. Nothing to install.

OPTION 2

Your VPC

Runs inside your own AWS or Azure account.

Model data follows the provider route you select.

OPTION 3 · CERTE

On-premise

Runs on customer infrastructure.

Installed with supplier-provided Helm charts and guidance.

> The platform Order Form covers the licence and managed service.

Managed Agents › Tools

Tools: who runs them

Code tools run in orq.ai. Function tools run in your process. MCP tools run on a server you register.



CODE TOOL · RUNS IN ORQ.AI

Python registered in orq.ai. One request runs the whole loop.

USE IT WHEN

The logic can live in orq.ai.



FUNCTION TOOL · RUNS IN YOUR PROCESS

Your code runs it and sends the result back to the model.

USE IT WHEN

The tool must reach your own database or private API, or needs your own approval step.



MCP · RUNS ON AN MCP SERVER

Register the server in orq.ai. Agents get its tools through the MCP Gateway.

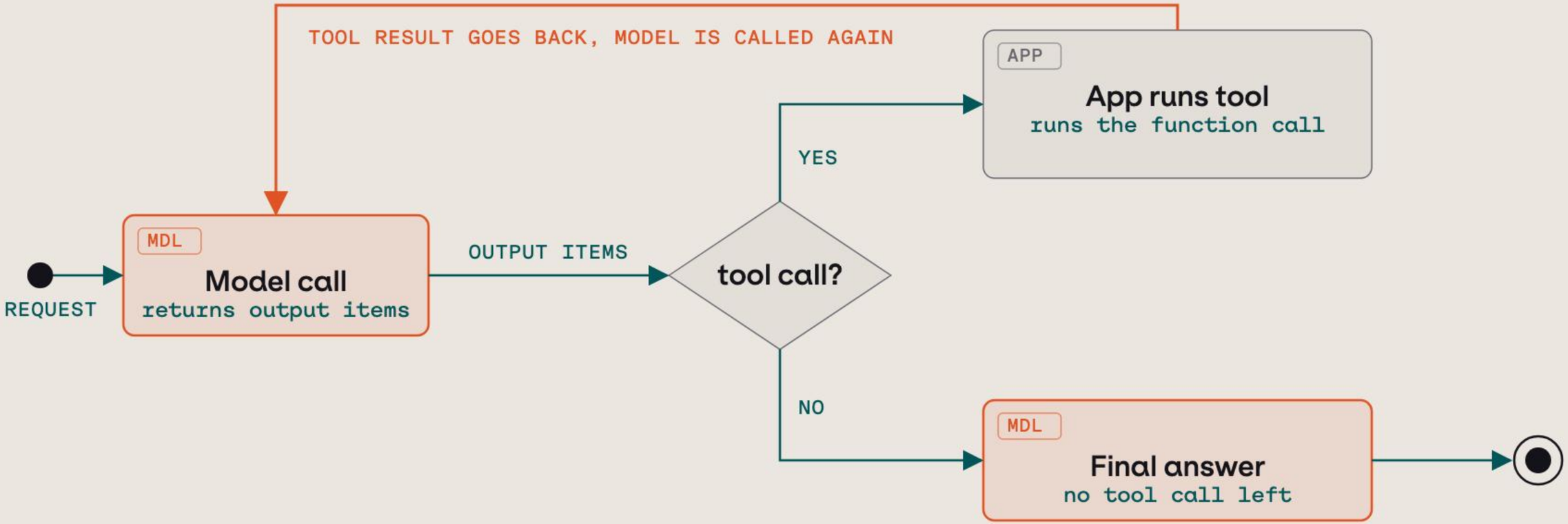
USE IT WHEN

The tools already exist as an MCP server, yours or a vendor's.

Managed Agents › Agents

The Responses tool loop

The model asks, your app runs the tool, and the result goes back until no tool call is left.



Local app owns the loop and resends all items with `store: false`.

Who owns the loop

Same tools on every path. What changes is who runs the loop and holds state.

APP CODE

YOUR APP Calls the model	→	YOUR APP Runs the tool	→	STATE Resend full items `store: false`

LANGGRAPH

GRAPH Calls the model	→	YOUR CODE Supplies the tools	→	STATE Graph carries it between steps

MANAGED AGENT

ORQ Runs the model	→	YOUR APP Runs function tools	→	STATE Stays with orq `previous_response_id`

All three use the workshop model from `.env`. Gateway routing and tracing apply; request controls such as cache apply when configured.

AI Gateway › MCP Portal

MCP vs MCP Gateway

MCP is the tool protocol. The gateway controls access to it.

MCP SERVER · DIRECT

CLIENT
Calls the server
Server-side credentials

→

MCP SERVER
Every tool exposed
No guaranteed logs

With the MCP Gateway

MCP GATEWAY · ONE GOVERNED URL

CLIENTS
One bearer key each
Named caller

→

GATEWAY
Allow-list per server
Every call logged, denied too

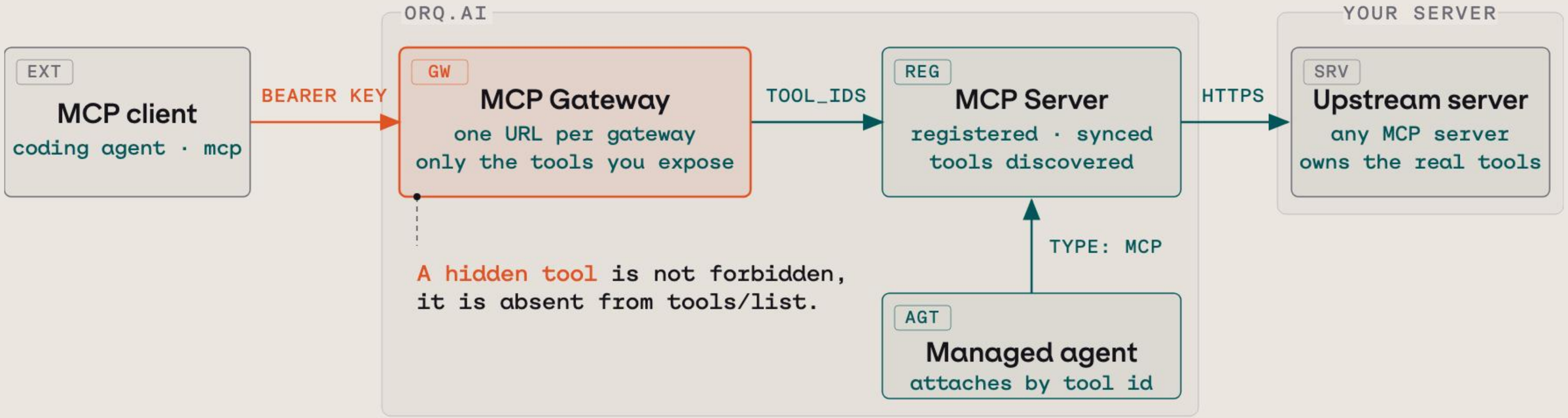
→

MCP SERVER
Only chosen tools
Rest stay hidden

● AI Gateway › MCP Portal

MCP: the trust chain

The client calls the gateway, which calls the registered MCP server.



AI Gateway › MCP Portal

Why a gateway sits in front

External MCP clients use the Gateway; managed agents attach to the registered Server directly.

zsh

```
❯ orq mcp-gateways list-tools <gateway-id> -o json
exposed name ← server_key/upstream_tool_name
tool_a          ← my-server/tool_a
tool_b          ← my-server/tool_b

# call a hidden tool through the gateway
MCPError: tool "hidden_tool" not found
```

Gateway clients see only exposed tools. Hidden tools are absent from `tools/list`; a direct call returns `not found`.

AI Gateway › Models

Choose the model before anyone builds

Catalog filters help you compare. Enforcement and project access decide who can call a model.

1
FILTER
Narrow the list
Location, provider, price, intelligence.

→

2
COMPARE
Europe + ZDR
Check the route and its access policy.

→

3
ENABLE
Approved models
Workspace-wide; default All projects.

→

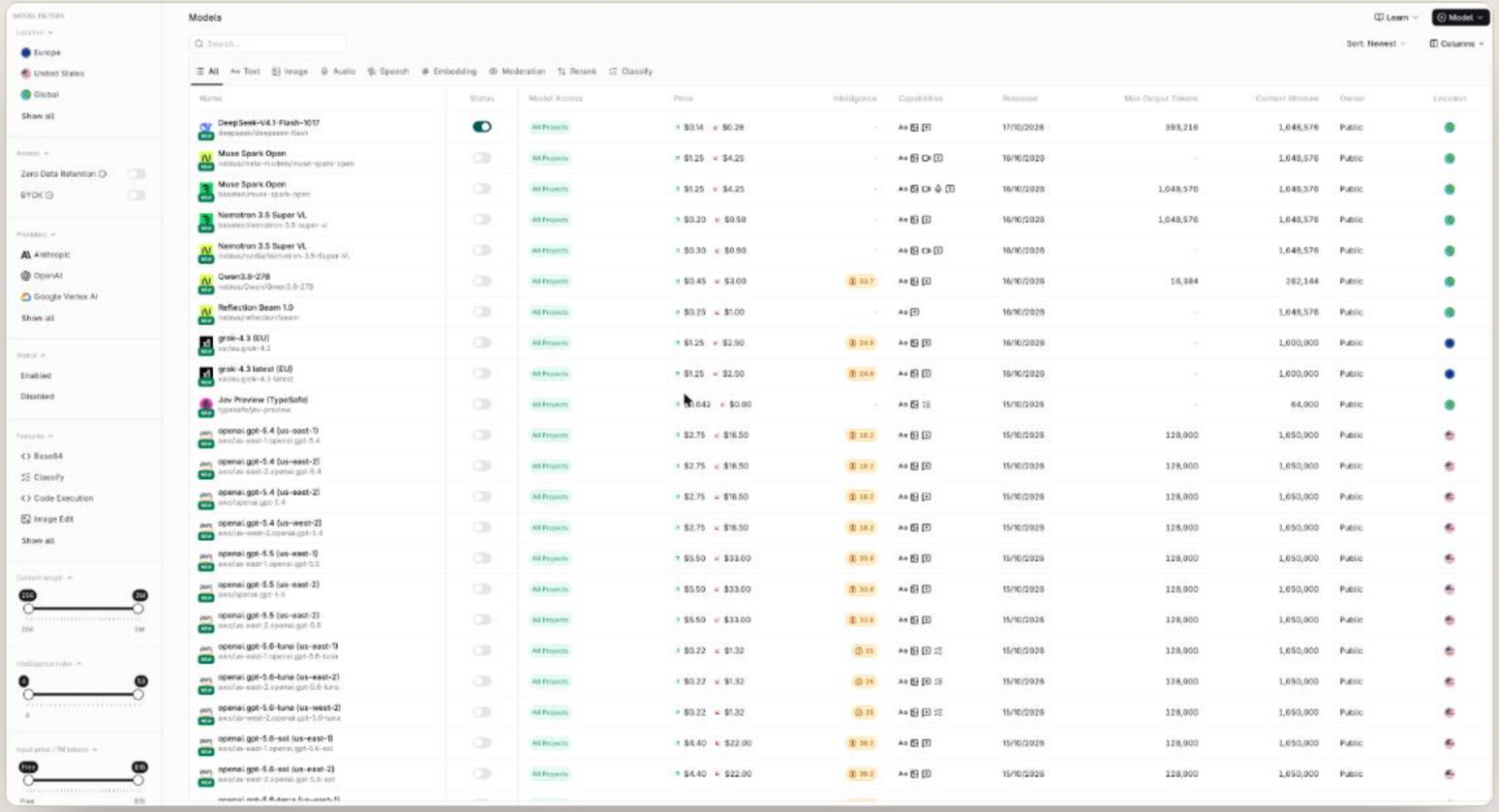
4
ENFORCE + SHARE
The real boundary
Enforce enabled models; Custom access, Certe test project only.

> Provider ZDR and orq.ai trace retention are separate controls.

● AI Gateway › Models

Model Garden: the full catalogue

Filters on the left, one row per model, with price, intelligence index, capabilities and location.

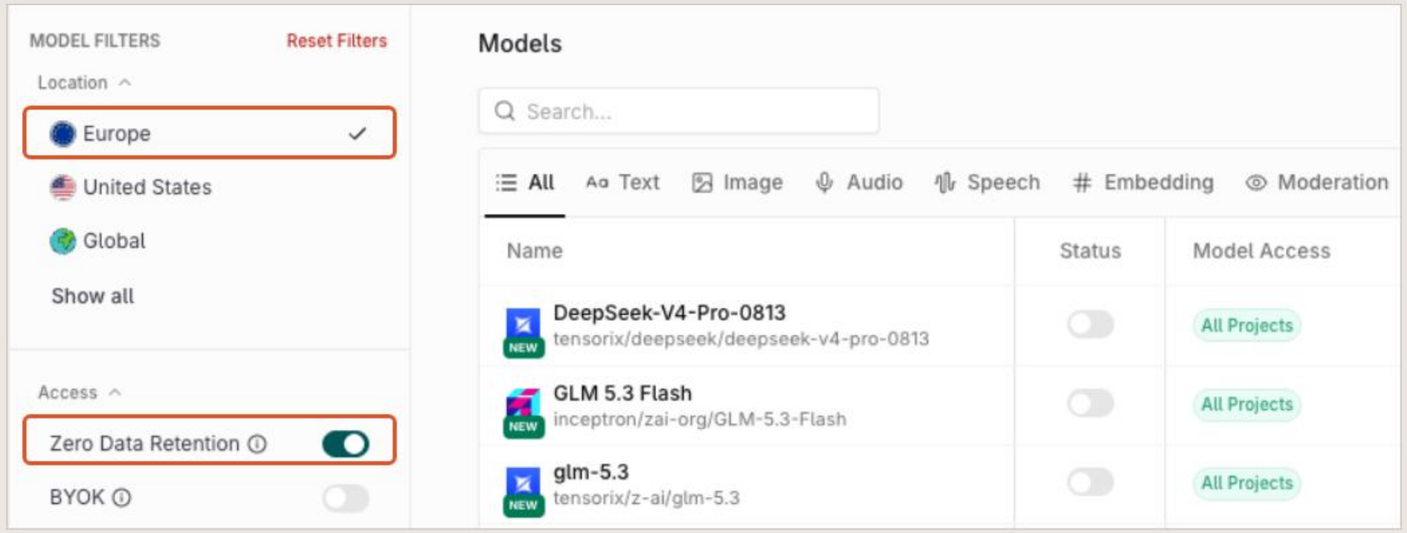


STUDIO · AI GATEWAY · MODELS · DEMO WORKSPACE

AI Gateway › Models

Model Garden: Europe and ZDR

These filters narrow the catalogue. They do not restrict model calls.



STAGING · MODELS · EUROPE + ZDR · 17 OCT 2026

Prompt Management › Playground

Playground: compare configurations

The same question on two models. Check prompts and settings before comparing answers.

A · GPT-6 LUNA · SYSTEM PROMPT SET · STAGING · SYNTHETIC DEMO



B · GPT-5.4-NANO · SYSTEM BLOCK EMPTY · STAGING · SYNTHETIC DEMO



AI Gateway › Models

Prove the model access boundary

Test workspace enforcement and project sharing independently.

CHECK	ACTION	EVIDENCE TO CAPTURE
Approved route	Call an enabled model from the test project.	ALLOW Successful response and its trace.
Unenabled model	With workspace enforcement on, call a model not enabled for the workspace.	DENY HTTP error; trace if recorded.
Project access	Call an enabled model from a project excluded by its Custom access control.	DENY HTTP error; trace if recorded.
Configuration	Read back workspace enablement, enforcement, and Custom project sharing.	Settings views showing each control.

> A catalogue filter or Playground comparison does not prove access enforcement.

AI Gateway › Models

Exercise: limit the test project to Europe and ZDR

Apply the filters, restrict the pool, then try to step outside it.

15 min practical + 5 min questions

1. Open **AI Gateway › Models** and switch on the **Europe** location and **Zero Data Retention** filters.
2. Note which models remain in the pool.
3. Restrict the test project to that pool.
4. Call a model outside the pool from the Playground or the API.

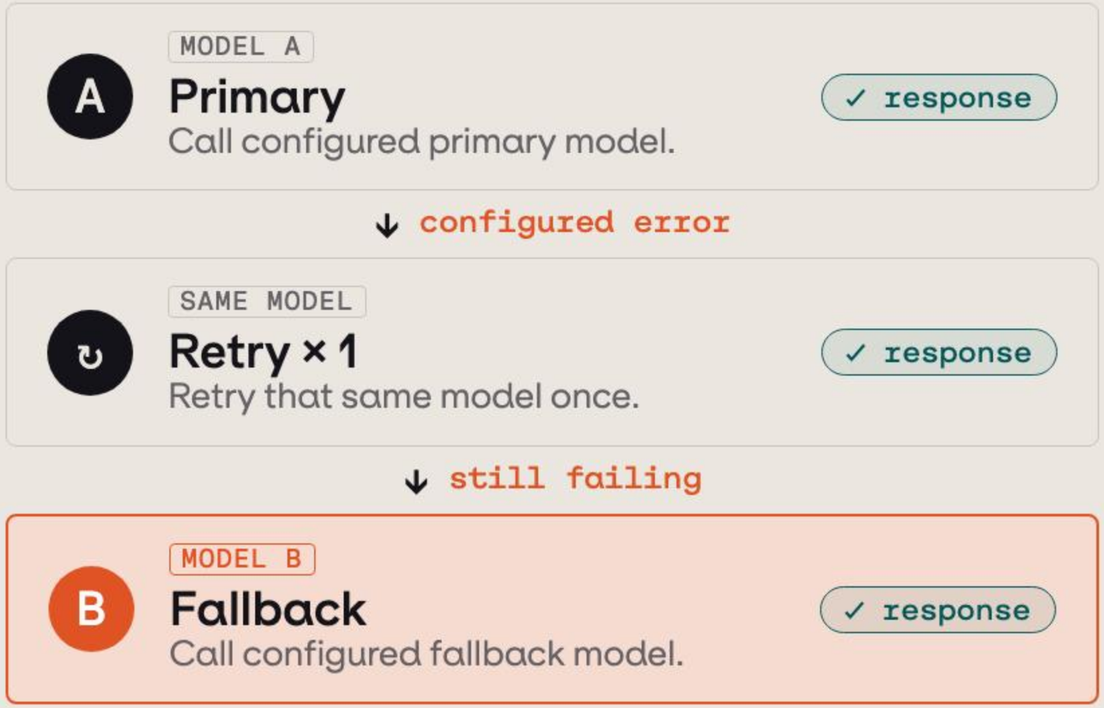
DONE WHEN

The call is denied and you can name the model it asked for.

AI Gateway

AI Gateway: key concepts

Four gateway controls and the span that records them.



01 Cache
An exact repeat is served from cache, avoiding provider cost.

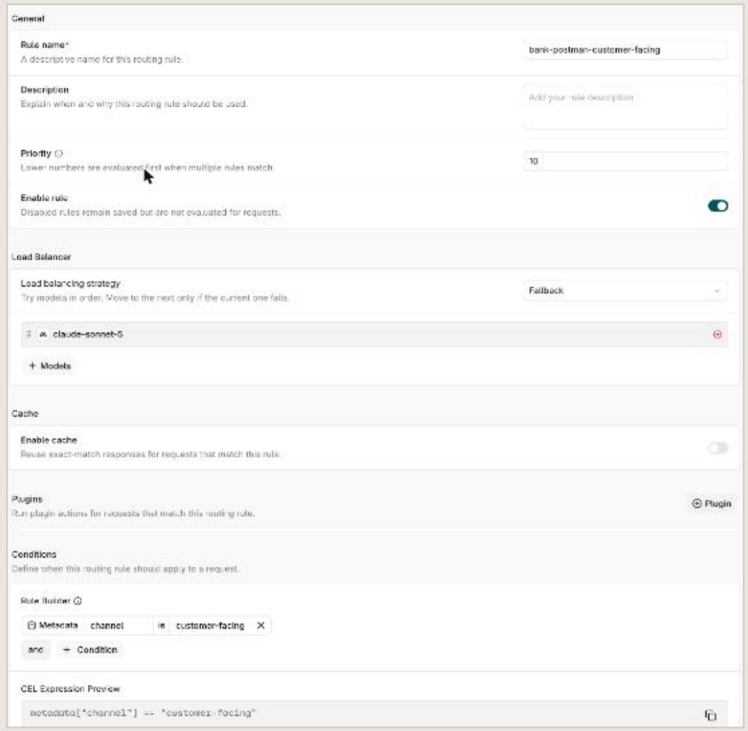
02 Load balancing
Weights like 50/50 set proportions; short runs may not split evenly.

03 Span
One step in a trace, such as a model call or a fallback decision.

AI Gateway › Routing Rules

Routing rules: key concepts

The first matching active rule takes precedence over the requested model.



STUDIO · EDIT ROUTING RULE · DEMO WORKSPACE

Priority
Lower numbers are evaluated first.

Enable rule
Disabled rules are saved, never evaluated.

Load balancing strategy
Fallback, latency, weight or round-robin.

Cache
Reuse exact-match responses.

Plugins
Extra actions on matching requests.

Conditions
Headers, identity, metadata, model or project, as a CEL expression.

Live check: match an active rule; inspect the chosen target in its trace.

● Settings › Workspace

Workspace governance: key concepts

Review the workspace, project and Enterprise controls with your admin.

WORKSPACE Set once by admins · applies to every project

SECURITY

SSO · OIDC / SAML ★ | Domain verification ★ | IP allowlist ★

MEMBERS AND ROLES

Admin | Developer | Researcher

PROJECTS Work and API keys are scoped here

PROJECT A
Agents and resources
API key · this project only

PROJECT B
Agents and resources
API key · this project only

PROJECT C
Agents and resources
API key · this project only

★ Enterprise plan or add-on

Settings › API Keys

API keys: modes and service keys

A standard API key belongs to one project; choose its permission mode within that project.

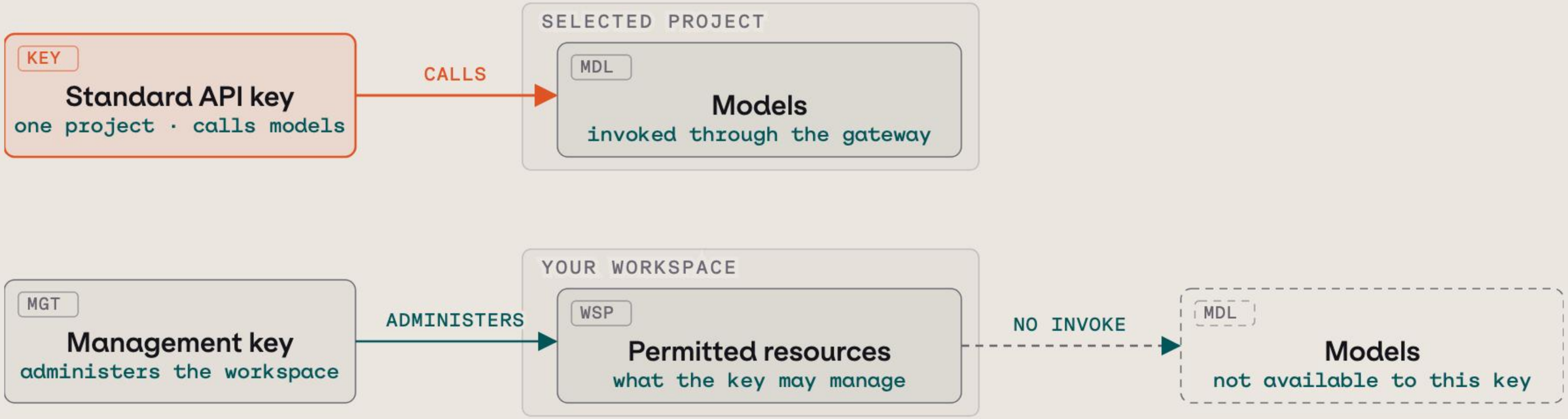
CREDENTIAL / MODE	SCOPE	USE
ALL	Selected project	All supported resources and actions in that project.
READ_ONLY	Selected project	Read access in that project; no writes.
RESTRICTED	Selected project	Per-resource permissions for only the access an app needs.
Management Key	Workspace	Separately permissioned workspace administration; cannot invoke models.

> Service account keys are created by admins for jobs that should outlive an individual user. User keys are revoked when that user is removed.

● Settings › API Keys

API keys: two credentials, two jobs

One calls models, one administers the workspace.



Settings › API Keys

API keys: do and do not

Agree on these defaults with IT before keys are handed out.

DO

- **Prefer restricted keys**
  Grant only what the app needs.
- **Use one key per app and project**
  A leak then has a smaller blast radius.
- **Use service account keys for CI**
  They outlive any one person.
- **Set a budget where appropriate**
  A budget is a spend control, separate from permissions.

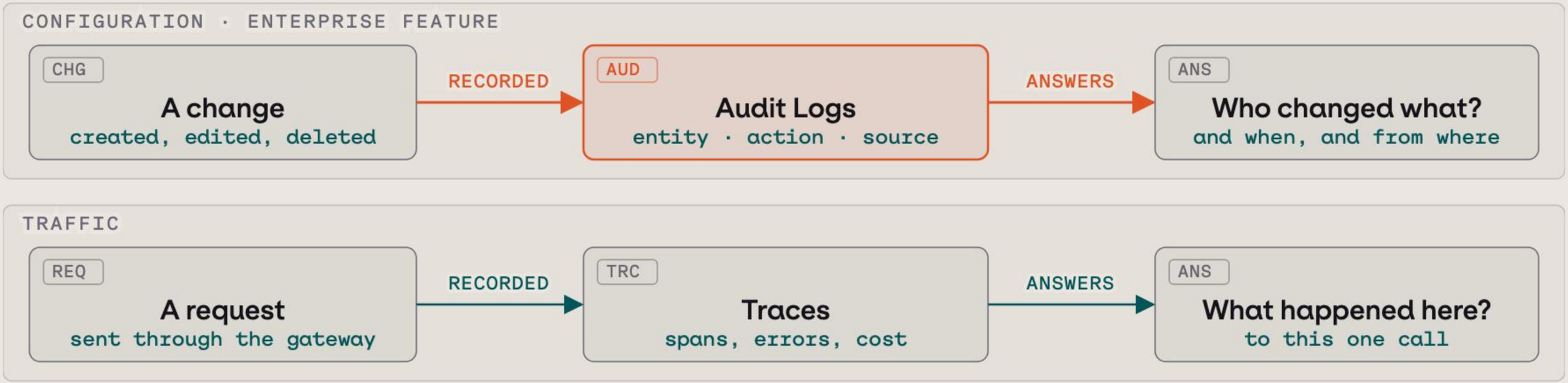
DO NOT

- **Share one broad key across teams**
  Use separate keys so access and usage can be managed.
- **Put keys in code or chat**
  Use the secret store.
- **Use a personal key for a shared job**
  It stops when that person leaves.
- **Assume the selected grant was stored**
  Read back the key's project and permissions.

Settings › Audit Logs

Audit logs: who changed what

Audit Logs record changes to the workspace. Traces record what happened to requests.



Retention follows the workspace retention. A rejected request is not an audit event: look for it in Traces.

● Settings › API Keys

Exercise: restricted key and audit log

Create a restricted key, prove its limits, then find the record of who made it.

⏱ 15 min practical + 5 min questions

1. Create a **restricted** project key with only the permissions the test needs.
2. Use it for one allowed read.
3. Attempt a write it is not allowed to make.
4. Open **Audit Logs** and find the entry left by creating the key.

DONE WHEN

The write is refused and the audit entry shows who created the key and when.

AI Gateway

Budgets and alerts

A budget caps spend. The gateway checks it before the provider is called.

BEFORE THE PROVIDER

REQUEST
New call

→

BUDGET CHECK
Every matching budget, independently
Workspace · Project · Identity · API key · Provider · Model

→

PROVIDER
Model call

BLOCKED Any exhausted limit, or the requests-per-minute cap, stops a new request here.

AFTER THE CALL FINISHES

IN FLIGHT
May complete

→

ACCOUNTING
Usage recorded

→

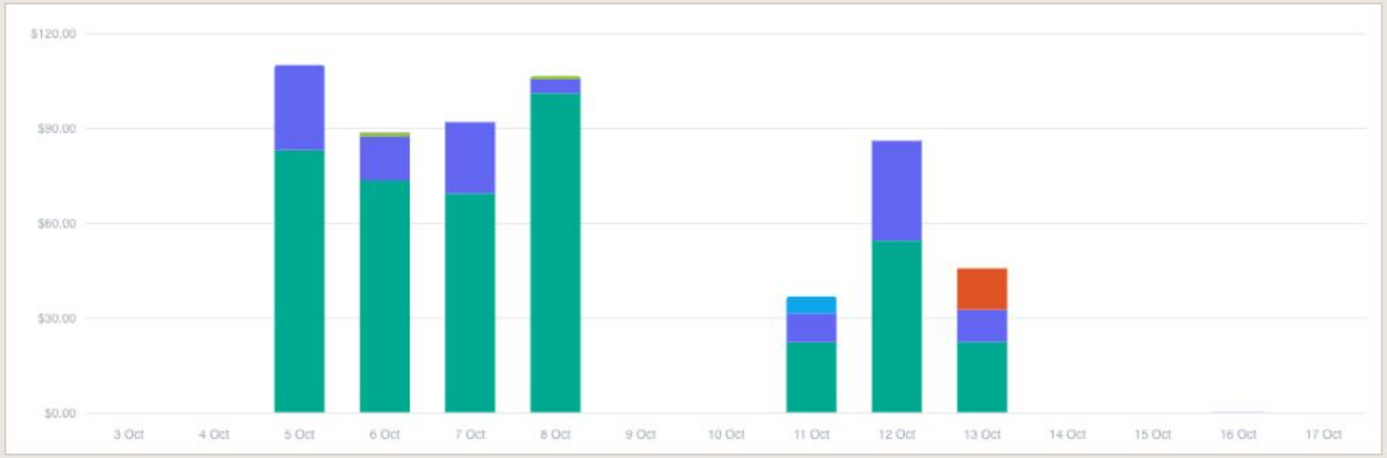
COST ALERT
Notifier fires

The requests-per-minute cap blocks requests; it does not trigger the cost alert.

Observability

Usage reporting: key concepts

The numbers behind the Studio views, and a cost alert on top.



STAGING · ACTIVITY · COST BY MODEL (OPUS 5.5, SONNET 5.5)

01 Reporting
`genai.usage` groups requests, failed spans and cost by model or key.

02 Error rate
Query `genai.error_rate` separately; counts are failed spans.

03 Cost alert
Checked on its interval; a notifier sends the trigger.

04 Check interval
Intervals: 5 min, 15 min, hourly, daily.

05 Lag
Reporting can lag; check again after a spike.

Observability › Alerts

Usage reporting: what alerts see

Alerts only see traces that reached a project.

PROJECT SIGNALS

Alerts can watch attributed project traces, including error and error-rate signals.

CHECK THE DENY PATH

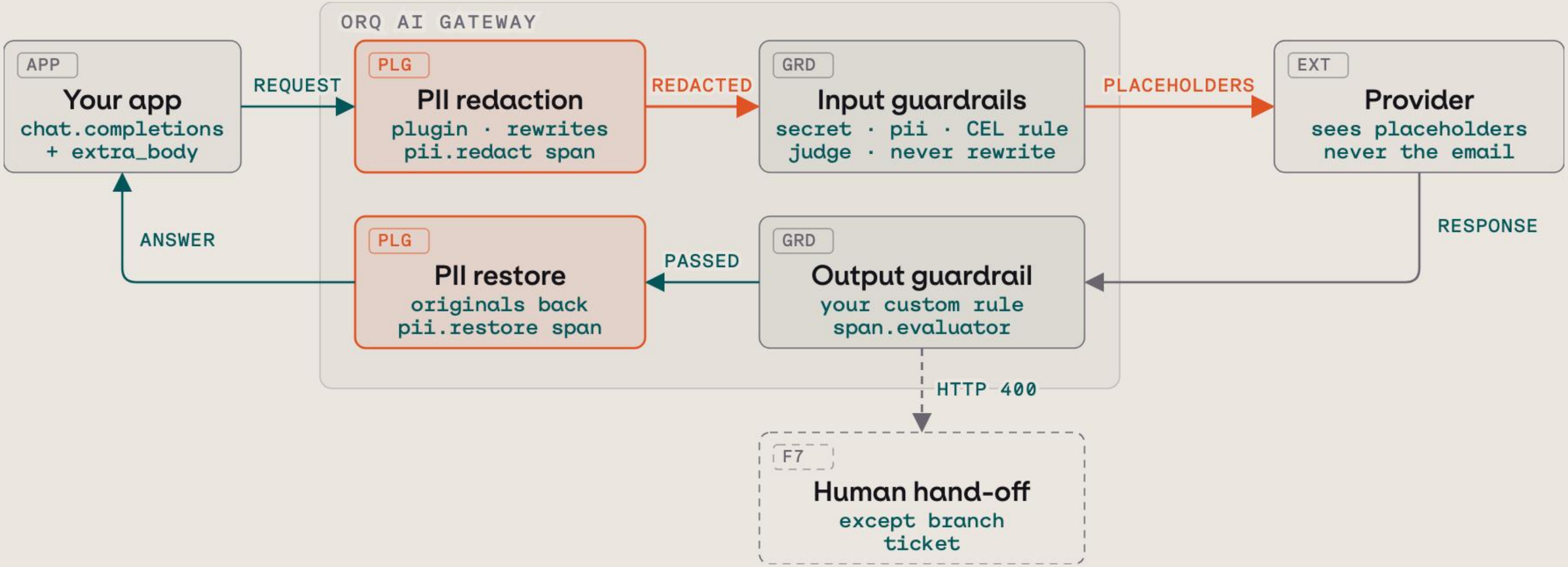
Some requests rejected before project attribution may be absent from project alerts.

Verify the specific rejected request in Traces; do not assume every timeout or guardrail block is invisible.

AI Gateway › Guardrail Rules

Guardrails and PII: the request path

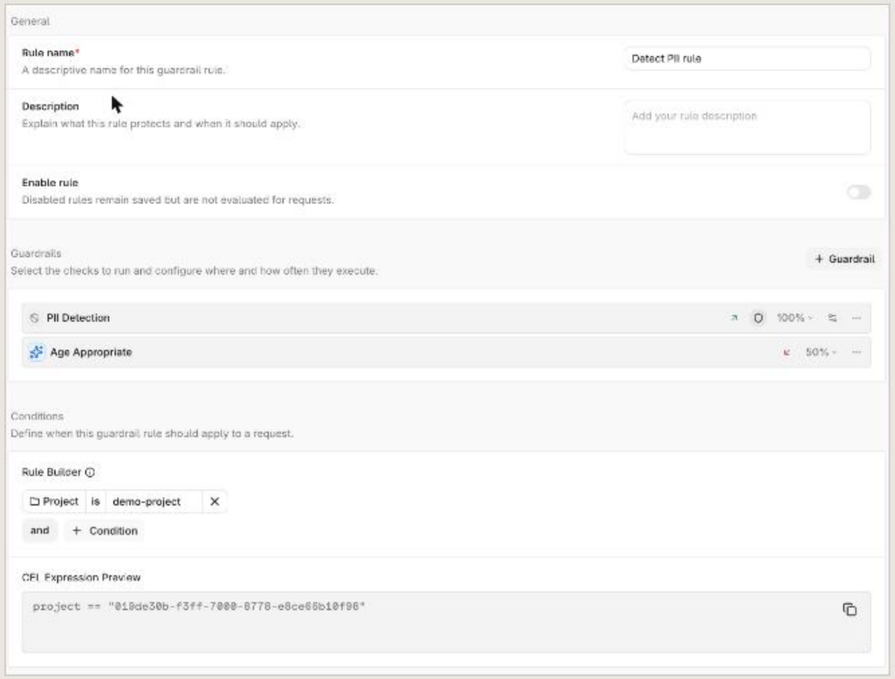
PII is replaced before the provider call and restored in the reply.



AI Gateway › Guardrail Rules

Guardrail rules: key concepts

A rule attaches checks to every request that matches its conditions.



STUDIO · EDIT GUARDRAIL RULE · DEMO WORKSPACE

Enable rule

Disabled rules are saved, never evaluated.

Guardrails

The checks to run, such as PII detection or an LLM-based check; a rule can hold several.

Where and how often

Each check has its own execution settings: which side it applies to and the share of requests it runs on.

Conditions

Headers, identity, metadata, model or project, as a CEL expression.

Live check: send a matching request and inspect the guardrail result in its trace.

● AI Gateway › Guardrail Rules

Exercise: guard a request and redact PII

Block or mask sensitive data at the gateway, then prove it in the trace.

⏱ 15 min practical + 5 min questions

1. Create a rule in **AI Gateway › Guardrail Rules** with the PII Detection guardrail on input.
2. Switch on PII redaction for a routing rule, including Dutch identifiers such as a BSN.
3. Send a request containing a fictitious BSN.
4. Open its trace and find the guardrail result and the redaction span.

DONE WHEN

The provider only received placeholders, and the caller got the real values back.

Observability › Traces

Traces and threads: key concepts

Search by thread or identity, then open a trace to read each step.

Name	Duration	Tokens	Cost
ws-refund-agent	1.69s	897	$0.000147
ws-refund-agent	0.96s	789	$0.000129

STUDIO · TRACES WITH DURATION, TOKENS AND COST

01 **Trace**
A gateway call starts or joins a trace; `traceparent` groups a turn.

02 **Span**
One step in a trace: a model or tool call.

03 **Cost and tokens**
Shown per request in the traces list.

04 **Request fields**
Send identity, thread and app name; they become trace filters.

05 **Guardrail block**
Root span turns red: `orq.guardrail.action=block`.

Observability › Traces

Traces: who writes which span

Your app writes the root and tool spans. The gateway writes each model call.

TRACE ITEM	WRITTEN BY	WHAT IT REPRESENTS
Root span	Your app	The full agent turn; propagate `traceparent` to group calls.
Model-call span	Orq gateway	Each routed model request, with usage and outcome.
Tool span	Your app	Instrument tool execution and attach it to the same trace.
Identity and thread	Request fields	Attribution and conversation filters; the sample app sends identity as `X-ORQ-IDENTITY-ID`.
App name	Request metadata	Optional metadata filter for app-level attribution.

AI Gateway › MCP Portal

MCP Portal: key concepts

Clients call the gateway. The gateway calls the server.

Name	Servers	Exposed tools	Access
ws-refund-gateway	R	2	All Projects

STUDIO · MCP GATEWAYS

Server and gateway

The server is your upstream registration. The gateway is the public endpoint clients use.

Selected tools

Only picked tools are exposed. Hidden-tool calls fail and are logged as denied.

Read-only filter

Keeps read-only tools; unmarked tools disappear.

AI Gateway › MCP Portal

MCP Portal: controls and outcomes

OAuth for interactive clients; the workshop uses a bearer key.

CONTROLS

Client authentication

Headless clients can send a bearer API key instead.

Egress policy

Blocked domains stop calls to those servers. Other servers keep working.

Runtime limit

Set request and tool timeouts; inspect failed calls in Gateway Overview.

OUTCOMES

Overview tab

Calls by outcome, a tools leaderboard and P95 latency.

Outcomes and traces

The Gateway Overview records successful and denied calls. A managed-agent trace shows its MCP server tool span.

● AI Gateway › MCP Portal

Exercise: expose one tool, hide another

Put a gateway in front of a server and watch it enforce what you expose.

⏱ 15 min practical + 5 min questions

1. In **MCP Portal**, add the suggested server (type HTTP, authentication **None**) and let it sync.
2. Create an MCP gateway, link the server and expose **one** tool.
3. Connect an MCP client to the gateway URL.
4. Call the exposed tool, then ask for the hidden one.
5. Find both calls in Traces.

SUGGESTED SERVER · NO AUTHENTICATION

MCP Playground Complex Server

`https://mcpplaygroundonline.com/mcp-complex-server`

Dummy business tools, nothing is stored. Expose `analyze_data`, hide `process_order`.

DONE WHEN

The hidden tool is logged as denied, not silently missing.

Managed Agents › Knowledge Bases

Knowledge bases: key concepts

Five settings decide what a search returns.

Name	Description
ws-refund-policy	Lumen Goods refund policy (workshop)
ws-refund-policy-large	Lumen Goods refund policy (workshop, module 09 ...

STUDIO · KNOWLEDGE · KNOWLEDGE BASES

01 Chunking
Splits files into passages; size and overlap shape what search finds.

02 Embedding model
Embeds chunks and queries. After a model change, re-embed.

03 Search type
Vector: meaning. Keyword: exact words. Hybrid: both (workshop base).

04 Reranking
Needs an enabled model; look for a rerank score.

05 Attach to an agent
The agent also needs the knowledge search tools.

Knowledge bases: search modes

Same question, three modes. Compare scores only within one mode.

SEARCH MODE	HOW IT MATCHES
Keyword	Matches exact terms and phrases.
Vector	Matches semantic similarity.
Hybrid	Combines keyword and vector results.

Managed Agents › Knowledge Bases

Exercise: compare search modes

Ask the same question three ways and read the chunks that come back.

10 min practical + 5 min questions

1. Open the prepared knowledge base in **Knowledge Bases**.
2. Run one question in vector, keyword and hybrid search.
3. Compare the returned chunks and their scores.
4. Change the chunking and search again.

DONE WHEN

You can say which mode returned the best chunk for which kind of question.

Managed Agents › Agents

Managed agents: the API

Function tools return a call for your app to execute, then continue the Responses turn.

FUNCTION TOOL LOOP

01 APP	02 AGENT	03 APP	04 APP
Send	**Call**	**Execute**	**Continue**
App sends input to the managed agent with its configured model.	Agent returns a function call to the app.	App executes the function and returns its result.	App continues the turn until a final message.

HOSTED CODE TOOL

01 APP	02 HOSTED	03 HOSTED	04 AGENT
Send	**Run**	**Result**	**Final**
Same input as above.	Agent runs code in the hosted environment.	Hosted environment returns the result.	Agent returns a final message.

> Return `function_call_output` with the matching `call_id`; continue with `previous_response_id`.

AI Gateway

Gateway and CLI: key concepts

Two ways in, one set of traces and budgets.

app.py

```python
from openai import OpenAI

client = OpenAI(
    base_url=ROUTER_URL,
    api_key=ORQ_API_KEY,
)
```

SDK · CHANGE THE BASE URL AND THE CREDENTIAL

01 **Base URL and credential**
Check model IDs and endpoint features for your SDK.

02 **Anthropic Messages API**
Supported format; confirm your model and features.

03 **Request metadata**
App name can be metadata; identity is a separate field.

04 **`orq launch`**
Session-only wiring; persistent setup uses **`orq connect`** (next slide).

05 **MCP server and Skills**
Wired into this repo with **`orq setup`** (next slide).

Terminal: four commands

The CLI for scripts and technical users. MCP for plain-language work.

zsh

```
❯ orq setup --local --capability gateway
  # wire MCP and Skills into this repo
❯ orq connect --local --dry-run
  # inspect agent wiring, change nothing
❯ orq launch claude
  # start Claude Code through the gateway
❯ orq traces conversation <trace_id>
  # read one trace as a conversation
```